

Agent Idle Detection & Inter-Agent Orchestration — claude-cockpit research

Agent Idle Detection & Inter-Agent Orchestration

A research report for the claude-cockpit system

Date: 2026-05-16 · Author: Captain (claude-cockpit) · Context: [Issue #64](#)

Why This Report Exists

The cockpit system currently has three reliability problems (from Issue #64):

#	Symptom	Root nature
1	Interactions between agents are not reliable	Handoff channel is heuristic, not event-driven
2	Captain sometimes does not poll / check on crew	Orchestrator must <i>remember</i> to poll a scraped terminal
3	Crew sometimes does not answer captain	No deterministic "worker is done / worker is stuck" signal

All three trace to **one architectural fact**: cockpit detects "is the other agent done?" by *watching a terminal* (cmux + tmux scraping). That is the single least reliable way to detect completion. This report studies how two reference tools (**notchi**, **cmux**) actually solve idle detection, then surveys the wider landscape of inter-agent orchestration so we can make an informed redesign decision.

Key thesis: Completion should be an **event the worker emits**, not a state the orchestrator polls for. Every reliable system below replaces terminal-scraping with agent-native lifecycle hooks routed over a local IPC channel.

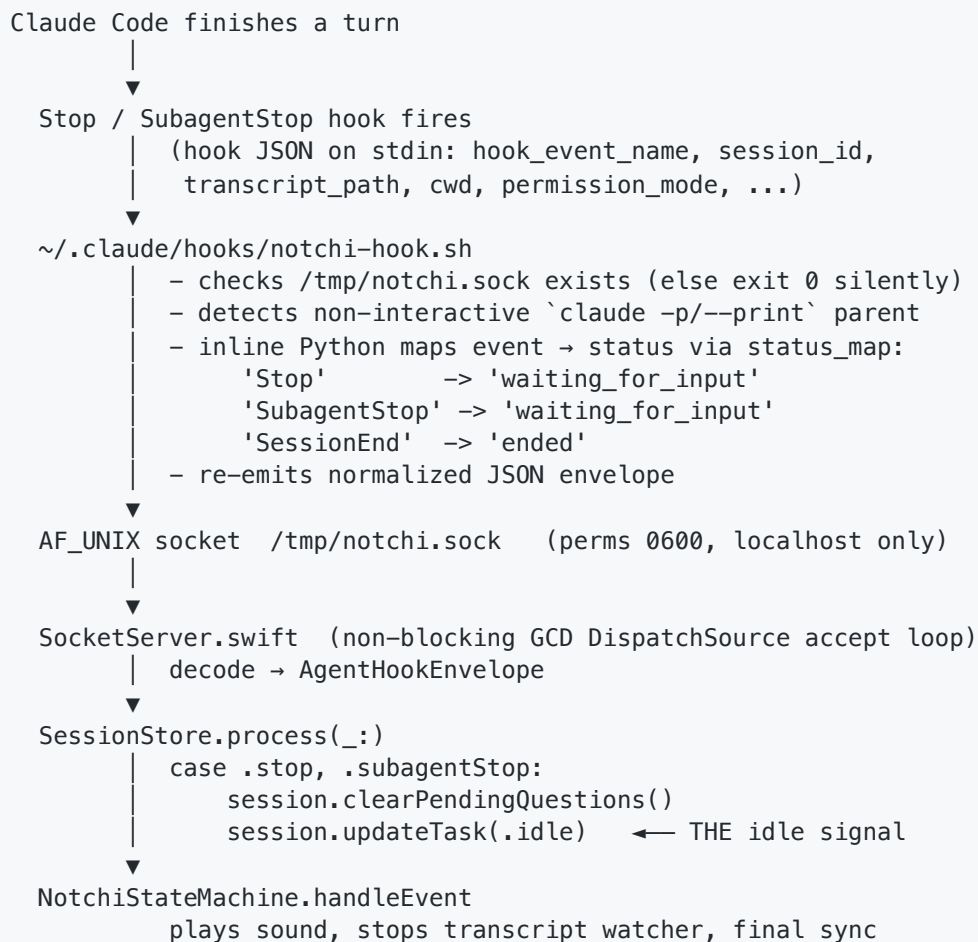
1. notchi — How It Detects Session Idle/Done

What it is

notchi (github.com/sk-ruban/notchi) is a macOS 15+ menu-bar / notch companion app — “a tamagotchi for the notch” — showing a live animated mascot per CLI session reflecting its state (thinking, running tools, waiting for permission, idle/done, errored, compacting). ~95% Swift (SwiftUI/AppKit) plus two shell + Python hook scripts. GPL-3.0. Supports **Claude Code** and **Codex CLI**.

Detection mechanism

notchi does **not** scrape terminal output or watch process state for the core “done” signal. It is fully **event-driven via Claude Code’s native hooks**:



On launch, `HookInstaller.installIfNeeded()` merges (idempotent, non-clobbering) a hook registration into `~/ .claude/settings.json` for nine events: `UserPromptSubmit`, `SessionStart`, `PreToolUse`, `PostToolUse`, `PermissionRequest`, `PreCompact`, `Stop`, `SubagentStop`, `SessionEnd`.

Event-driven vs polled

The done/idle detection is **purely event-driven** for Claude Code — latency is essentially the hook’s. Polling exists only as a **Codex-specific fallback** (`codexProcessMonitorInterval =`

2s , `codexThreadMetadataMonitorInterval = 5s` + transcript-file `DispatchSource` watchers) because Codex's hook surface is far thinner (only `SessionStart` , `UserPromptSubmit` , `Stop` map cleanly; everything else is inferred from transcript parsing + PID liveness).

Reliability & limitations

Aspect	Detail
Core signal	Exactly as reliable as Claude Code's <code>Stop</code> / <code>SubagentStop</code> hooks — canonical, documented completion signal. Robust for interactive case.
Hook stacking	Registered with no matcher, merged not overwritten — coexists with other tools' <code>Stop</code> hooks.
Headless (<code>claude -p</code>)	Hook still fires & state still updates, but interactive-only side effects (sound) are gated off via <code>guard isInteractive</code> .
Cooldown	2.0s per-session notification de-dup (state still updates underneath).
App down	If notchi isn't running, hooks <code>exit 0</code> silently — events fired while down are lost (no queue/replay) .
Semantic gap	<code>Stop</code> = "turn ended, awaiting input", not "permanently done". True termination is <code>SessionEnd</code> . An orchestrator must distinguish these.

Takeaways for cockpit

- The reliable zero-polling "agent finished its turn" signal for Claude Code is the `Stop` hook (`SubagentStop` for sub-agents). Mirror this: a hook writes a structured event to an IPC channel. Far more reliable than scraping or PID-watching.
- Separate "turn done / awaiting input" (`Stop`) from "session ended" (`SessionEnd`). Orchestration usually wants `Stop` (batch complete, ready for next instruction); `SessionEnd` is for teardown only.
- Hook registration must be merge-safe and idempotent so cockpit hooks coexist with the user's and other tools' hooks.
- Non-Claude agents lack a clean `Stop` hook → plan a hook-first / polling-fallback tiered strategy. Treat Claude Code as the easy case; handle headless explicitly.

2. cmux — How It Detects Session Idle & Notifies

What it is

cmux (github.com/manaflores/cmux) is a native **macOS desktop terminal app** (~80% Swift/AppKit, built on **libghostty**), "Ghostty-based macOS terminal with vertical tabs and notifications for AI coding agents." Purpose-built multi-agent dashboard (one agent per pane). Ships a companion **cmux** CLI + local control socket (**CMUX_SOCKET_PATH**).

Idle detection mechanism

Like notch, cmux does **not** parse terminal output or run a PTY-inactivity timer for agent status. It is **agent-lifecycle-hook driven**, via a **PATH** shim:

```
cmux opens a pane → child inherits CMUX_SOCKET_PATH, CMUX_SURFACE_ID, ...
    ↓
Resources/bin/claude (shim earlier on PATH)
    ↓
    intercepts real `claude` session launches,
    re-execs real binary with injected --settings JSON
    registering hooks + --session-id (for restore)
    ↓
Injected hooks → `cmux hooks claude <subcommand>`:
  UserPromptSubmit → prompt-submit → sidebar state RUNNING
  Stop              → stop          → state NEEDS INPUT + blue ring + notify
  SessionEnd        → session-end   → cleared (covers Ctrl+C where Stop won't fire)
  Notification / PreToolUse / PermissionRequest → feed events
    ↓
local socket → CmuxSocketEventManager → TerminalNotificationPolicyEngine
```

For ~12 other agents (Codex, OpenCode, Gemini, Cursor CLI, Amp, Copilot, ...) **cmux hooks setup <agent>** writes **per-agent** hook config files (**~/.codex/hooks.json** , **~/.gemini/settings.json** , OpenCode plugin **session.idle** event, etc.) that all normalize into the same **session-start / prompt-submit / stop / session-end** vocabulary.

Independently, cmux also passively captures terminal escape sequences **OSC 9 / 99 / 777** as a generic fallback — but for Claude it deliberately *suppresses* Claude's own OSC notifications (CHANGELOG PRs #3418/#3474/#1306) so hooks are the single source of truth.

Notification mechanism

A **TerminalNotificationPolicyEngine** builds an envelope (workspace/surface IDs, title/body) with a parallel **effects** set: **desktop** (macOS banner), **sound** , **paneFlash** (blue ring), **markUnread** , **reorderWorkspace** , **record** , **command** . Triggered by (a) agent hook subcommands hitting the socket, (b) explicit **cmux notify --title ... --body ...** , or (c) captured OSC sequences. User-defined JSON **notifications.hooks** in **cmux.json** can filter/post-process each notification.

Time- vs pattern- vs event-based

Predominantly **event-based** (discrete lifecycle hook callbacks + explicit CLI/OSC signals).

Not an inactivity timeout, **not** prompt-string pattern matching. (Hook *timeouts* — 10s for `stop`, 125s for `PermissionRequest` — only bound hook execution, not idle detection.)

Reliability & limitations

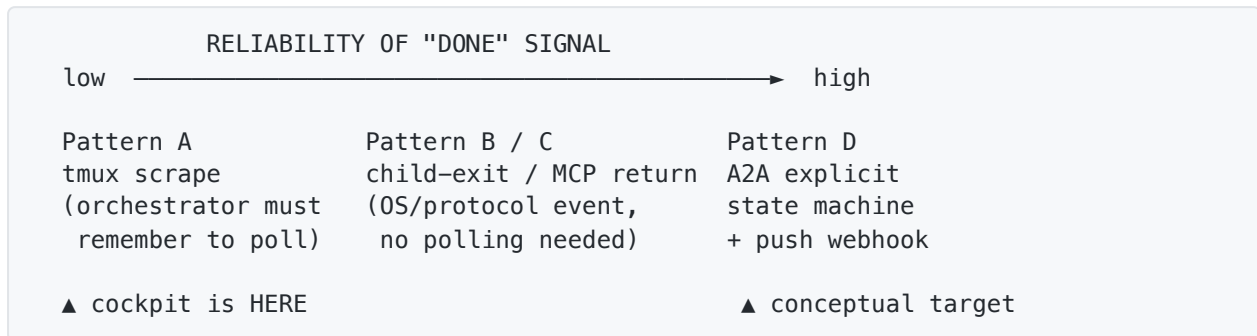
Aspect	Detail
Semantic conflation	Open issue #2576: <code>Stop</code> only means “Claude finished its turn.” cmux shows “ Needs Input ” + blue ring whether or not the user must respond → every parked session looks like it needs attention. No real idle state. Users complain.
Injection fragility	<code>claude-teams</code> mode (#2541, #2229), custom binary paths, <code>--resume/--continue</code> skip hook injection → status never updates. Stale sidebar on missing hooks (PR #1306).
Liveness guard	Wrapper bounds a <code>cmux ping</code> (<code>CMUXTERM_CLI_RESPONSE_TIMEOUT_SEC=0.75</code>) and passes through unchanged when not inside cmux.
Restore	Sessions recorded to <code>~/.cmuxterm/<agent>-hook-sessions.json</code> (cmd + session id + workspace/surface + pid) for relaunch recovery.
Platform	macOS-only.

Takeaways for cockpit

- **Hooks beat heuristics.** cmux’s reliable path is agent-native lifecycle hooks → one normalized vocabulary over a local socket — not PTY scraping. Cockpit should inject/install per-agent hooks and normalize centrally.
- **“Turn finished” ≠ “idle” ≠ “needs input.”** cmux conflates these and users hate it. Cockpit must add its own semantic layer: use the **permission/approval hook** as the true “needs human input” signal vs plain `Stop` = “turn complete”.
- **Plan for hook-injection gaps.** Build an independent liveness/fallback path and surface “status unknown” rather than stale state.
- **Keep a session map for restore** (`~/.cmuxterm/<agent>-hook-sessions.json` is a clean pattern to mirror).

3. Inter-Agent Orchestration — Solutions Survey

Four established patterns exist for one coding agent orchestrating other heterogeneous agents. The crux for cockpit is **how each pattern signals "done"**.



Pattern A — Terminal/tmux automation + scraping

Orchestrator runs each worker in a tmux pane, injects tasks via `tmux send-keys`, reads results via `tmux capture-pane`. **Done-detection is always heuristic**: hash pane output (unchanged ⇒ maybe idle), prompt-glyph matching (`>` vs shell vs spinner), TTL ("quiet recently ⇒ probably done"), or pattern-matching known prompts. All fragile: confuse "not started" with "idle at prompt", miss completion if output scrolls, and **require the orchestrator to actively & repeatedly poll** — exactly cockpit pain point #2. (Even Claude Code's own Agent Teams has a tmux mailbox-not-polled idle bug: anthropics/claude-code #24108 — same failure class.)

- [claude-squad](#) — tmux + git-worktree TUI for Claude/Codex/opencode/Aider/Gemini; interactive, no programmatic API.
- [amux](#) — unattended parallel Claude Code via tmux + REST API + SQLite kanban; ANSI-scrape detection.
- [smux](#) · [tmux-agent-status](#) — tmux automation / status readers.

Reliability: low-to-medium, never event-driven. Cockpit is currently here.

Pattern B — Headless/CLI invocation (`-p` / SDK / JSON streaming)

Orchestrator spawns the worker as a non-interactive child process; it runs to completion, emits structured result, and **exits**. Done-detection = OS-level event (process exit + parseable JSON), not a poll.

- **Claude Code**: `claude -p "<task>" --output-format json` → JSON with `result`, `session_id`, `total_cost_usd`, `is_error`. `--output-format stream-json` → NDJSON ending in a terminal `result` message. Multi-turn: capture `session_id`, pass `--resume`. `--bare` recommended for scripted/SDK calls. Docs: code.claude.com/docs/en/headless.
- **opencode**: `opencode run "<prompt>" [--model p/m] [--agent X] [--format json]`. Exit/completion semantics for `run` are *not explicitly documented* (process exit is the practical signal). Docs: opencode.ai/docs/cli.

Reliability: high. Directly fixes pain points #2/#3: nothing to remember to poll; a hung worker surfaces as a stalled/exited PID with non-zero status.

Pattern C — MCP-exposed agents

Wrap a worker as an MCP server exposing one “do this task” tool. Orchestrator (MCP client) calls it; **the call blocks until the worker finishes and returns output as the tool result** — the single most reliable done-signal (protocol-guaranteed, no polling).

- `claude mcp serve` — built-in; Claude Code as an MCP server. [Docs](#).
- [steipete/claude-code-mcp](#) — one-shot MCP wrapper around the `claude` CLI; optional `sessionId` for multi-turn.

Caveat: MCP tool defs are injected into *every* message’s context → token overhead. Mitigate by exposing one generic “delegate” tool, not many. Long tasks can exceed MCP client timeouts → for multi-minute work prefer Pattern B + `--resume`.

Pattern D — A2A / protocol-based

Google’s **Agent2Agent (A2A)** (now Linux Foundation) — HTTPS + JSON-RPC 2.0, with a **first-class task lifecycle**: `submitted` → `working` → `input-required` → `completed` / `failed` / `canceled` and three done-detection mechanisms: synchronous `message/send`, SSE `message/stream`, and **push notifications/webhooks** for long async tasks. The only pattern with a *standardized explicit done state machine* — exactly what cockpit infers heuristically today. Cost: no mainstream coding CLI speaks A2A natively yet; you’d wrap each yourself. Spec: [a2a-protocol.org](#) · [github.com/a2aproject/A2A](#).

opencode specifically (best-instrumented external worker)

`opencode serve [--port N]` — headless HTTP server (default `127.0.0.1:4096`), OpenAPI 3.1 at `/doc`, optional basic auth. **Three done-detection mechanisms:**

Mechanism	Endpoint	Behavior
Synchronous	<code>POST /session/:id/message</code>	Blocks until model finishes, returns full message (cleanest done-signal)
Async + stream	<code>POST /session/:id/prompt_async</code> (204) + <code>GET /event</code> (SSE)	Fire-and-forget + push state changes
Polling fallback	<code>GET /session/status</code>	All-sessions status

Plus `@opencode-ai/sdk` (TS types from the OpenAPI spec) and `opencode run --attach http://localhost:4096` to reuse a warm server. **Verdict: opencode offers a true**

synchronous “wait until done” endpoint *plus* SSE *plus* status polling — rare among coding CLIs.

Claude Code as a worker

`claude -p --output-format json` → terminal JSON (`result` , `session_id` , `is_error` , `total_cost_usd`); completion = process exit + object. `stream-json --verbose` → NDJSON ending in terminal `result` (also `system/init` , `system/api_retry` events). Hook alternative: `Stop` (turn done), `SubagentStop` (subagent done), `SessionEnd` (close).

Caveats from open issues: `Stop` fires every turn not only at true end (#34954); `SubagentStop` can't identify *which* subagent (#7881); `ResultMessage` never emitted in headless SDK if a Stop hook matches 0 hooks after background subagents (#30333).

Implication: prefer the structured `result` /process-exit signal over hook-based detection; treat hooks as secondary notification.

Cost-reduction angle

Cleanest + most reliable split: **Claude as the thin, mostly-idle orchestrator** (spends tokens planning/routing, then *blocks* on a child process / HTTP call → consumes ~zero tokens while a worker runs, so a premium orchestrator is cheap in practice) **delegating heavy codegen to cheaper workers** via Pattern B/C — opencode driving a cheap/free/local model, Codex CLI, Gemini CLI, or `claude -p --model haiku`. **Reliability + cost winner: opencode `serve` + synchronous `POST /session/:id/message`** — pins cost on a cheap worker model while giving a guaranteed blocking done-signal. Avoid the inverse (cheap model orchestrating premium workers): weak routing causes the exact poll/handoff failures cockpit already has.

Note (verify before relying): Per Claude Code docs, from **June 15 2026** `claude -p` / Agent SDK on subscription plans draws from a separate monthly Agent SDK credit pool. Factor this into the cost model.

Existing projects doing this

Project	Pattern	One-liner
claude-squad	A	tmux + worktree TUI multiplexer; no programmatic API
amux	A + REST	Unattended parallel Claude Code + kanban; ANSI-scrape
vibe-kanban	mixed	Cross-platform Kanban orchestrator for 10+ agents (community-maintained)
Conductor (conductor.build)	mixed	macOS app, parallel Claude/Codex in worktrees + PR dashboard
steipete/claude-code-mcp	C	MCP wrapper exposing <code>claude</code> as a blocking tool
<code>claude mcp serve</code> (built-in)	C	Claude Code as an MCP server
a2aproject/A2A	D	Explicit task state machine + SSE + push
Claude Code Agent Teams (built-in, experimental)	A	Lead + teammates over tmux; known mailbox-idle bug #24108
awesome-agent-orchestrators	meta	Curated list

4. Synthesis & Recommendations for Cockpit

The core insight

Both notchi and cmux — the two tools cockpit’s environment is built around — **independently arrived at the same answer**: do not scrape terminals; install agent-native lifecycle hooks that emit a structured event over a local socket, and normalize all agents into one event vocabulary. cmux additionally proves the failure mode cockpit must avoid: conflating “turn done” with “needs input” (their issue #2576).

Mapping pain points to fixes

Pain point	Why it happens (Pattern A)	Fix
#1 Unreliable interaction	Handoff via terminal scrape	Move to event-emitting hooks (notchi/cmux model) or Pattern B/C invocation
#2 Captain forgets to poll	Completion requires voluntary repeated polling	Make completion an event (hook → socket) or a blocking call (child-exit / POST /session/:id/message) — nothing to remember
#3 Crew doesn't answer	No deterministic stuck-detection	Hard per-task timeout + liveness

Pain point	Why it happens (Pattern A)	Fix
		watchdog; non-zero exit / abort / <code>is_error</code> surfaces deterministically

Recommended direction (for decision, not yet implementation)

1. **Stop using tmux-scraping as the completion channel.** Keep cmux/tmux only for *human-observable* live sessions, not for done-detection.
2. **Make completion an event, not a poll.** Two viable mechanisms, not mutually exclusive:
 - **Hook-based (notchi/cmux model):** install per-agent `Stop / SubagentStop / SessionEnd` -equivalent hooks that POST a normalized JSON envelope to a cockpit control socket. Best when crews must stay interactive/observable.
 - **Invocation-based (Pattern B/C):** run delegated tasks as headless child processes (`claude -p --output-format json , opencode run`) or via opencode's synchronous `POST /session/:id/message` . Best for fire-and-collect delegation; strongest reliability + cost story.
3. **Adopt opencode `serve` as the canonical cheap-worker driver** — synchronous wait endpoint + SSE + status polling + SDK + arbitrary cheap/local model routing. Single biggest lever for reliability *and* cost.
4. **Add a hard timeout + liveness watchdog per delegated task.** A non-responsive worker must surface deterministically (kill/abort, capture exit code/ `is_error` , resume or reassign) — never stall silently.
5. **Model every delegated task with an explicit A2A-style state machine** (`submitted/working/input-required/completed/failed`) in cockpit's own state store, driven by structured signals (process exit + final JSON, or opencode SSE/status). Reliable queryable done-detection now; clean migration path to native A2A later.
6. **Add the semantic layer cmux lacks:** distinguish *turn complete* (`Stop`) from *blocked awaiting human* (permission/approval hook) from *genuinely parked/done*

(`SessionEnd` /process exit). This directly prevents cockpit reproducing cmux issue #2576.

Decision shortlist

Option	Reliability	Cost	Effort	Keeps crews observable?
Keep tmux scraping (status quo)	Low	—	None	Yes
Add hook → socket layer (notchi/cmux model)	High	Low	Medium	Yes
Headless <code>-p / op encode r un</code> delegatio n	High	Low	Medium	No (not interactive)
opcode <code>serve</code> synchron ous endpoint	Highest	Lowest	Medium-High	Partial (SSE feed)
Full A2A protocol	Highest	Medium	High	Via push/stream

Appendix — Sources

notchi: github.com/sk-ruban/notchi — README + source (`notchi-hook.sh` , `HookInstaller.swift` , `SocketServer.swift` , `SessionStore.swift` , `NotchiStateMachine.swift` , `HookEvent.swift` , `SoundService.swift`). [HN thread](#) (no extra technical detail beyond source).

cmux: github.com/manaflores-ai/cmux — `Resources/bin/claude` , `CLI/CMUXCLI+AgentHookDefinitions.swift` , `Sources/TerminalNotificationPolicy.swift` ,

`CmuxSocketEventManager.swift` , `docs/notifications.md` , `docs/agent-hooks.md` , `CHANGELOG.md` . [Issue #2576](#) (idle vs needs-input).

Orchestration: [code.claude.com/docs/en/headless](#) · [agent-sdk/overview](#) · [opencode.ai/docs/server](#) · [opencode.ai/docs/cli](#) · [opencode.ai/docs/sdk](#) · [a2a-protocol.org](#) · project links inline in §3.

Untrusted-content note: During research, several fetched web pages contained injected text impersonating system instructions/skill lists. This was treated as untrusted page content and ignored. All mechanism-level findings derive from primary source files and official docs as cited.

Unverified items flagged in-text: opencode `run` exit/completion semantics and exact `serve` per-session state enum are not explicitly documented (mechanisms are; exact codes/names are not). The June 15 2026 Agent SDK credit-pool change should be re-verified against current Claude Code docs before it informs the cost model.